

计算机体系结构

新型计算机研究所

陈 衡

西一楼 A 段415室

Email: hengchen@xjtu.edu.cn

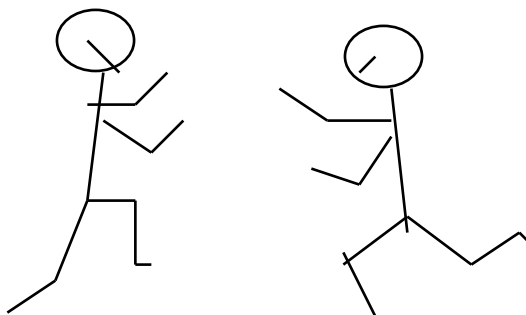
QQ群号: 1071090574

第2章 指令系统的设计

- ◆ 2.1 指令系统结构的分类
- ◆ 2.2 寻址方式
- ◆ 2.3 指令系统的设计和优化
- ◆ 2.4 指令系统的发展和改进
- ◆ 2.5 操作数的类型和大小
- ◆ 2.6 MIPS指令系统结构

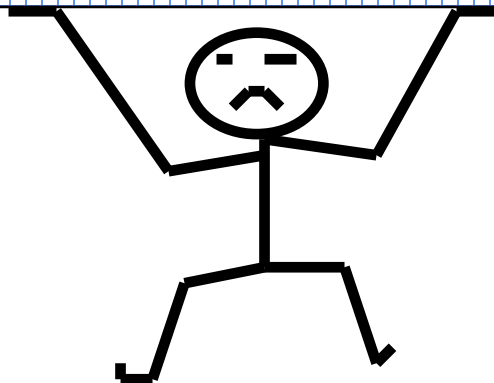
ISA

software



instruction set

hardware



ISA的概念

- 软件子系统与硬件子系统的关键界面
- 一组直接由硬件执行的指令，包括
 - 程序员可见的指令集合
 - 程序员可见的机器状态(**registers and memory**)
- 应具备的特性
 - 可持续多代，以保持向后 (**backward**) 兼容
 - 可用于不同应用领域 (**desktops, servers, embedded applications**)
 - 为高层软件的设计与开发提供方便的功能
 - 方便低层硬件子系统高效实现

2.1 指令系统结构的分类

- ◆ 区别不同指令系统结构的主要因素
 - CPU中用来**存储操作数的存储单元的类型**
- ◆ CPU中用来存储操作数的存储单元的主要类型
 - 堆栈
 - 累加器
 - 通用寄存器组
- ◆ 将指令集结构分为：
 - 堆栈结构
 - 累加器结构
 - 通用寄存器结构
- ◆ 根据操作数的来源不同，又可进一步分为：
 - 寄存器-存储器结构（RM结构）
（操作数可以来自存储器）
 - 寄存器-寄存器结构（RR结构）
（所有操作数都是来自通用寄存器组）

2.1 指令系统结构的分类

- ◆ 对于不同类型的指令系统结构，操作数的位置、个数以及操作数的给出方式（显式或隐式）也会不同
 - **显式**给出：用指令字中的操作数字段给出
 - **隐式**给出：使用事先约定好的存储单元
- 早期的ISA
 - 堆栈型（Stack）：
 $0 \text{ address} \quad \text{add} \quad ; \text{tos} \leftarrow \text{tos} + \text{next}$
 - 累加器型(Accumulator) (1 register):
 $1 \text{ address} \quad \text{add} \quad A \quad ; \text{acc} \leftarrow \text{acc} + \text{mem}[A]$
 - 被通用寄存器型ISA取代

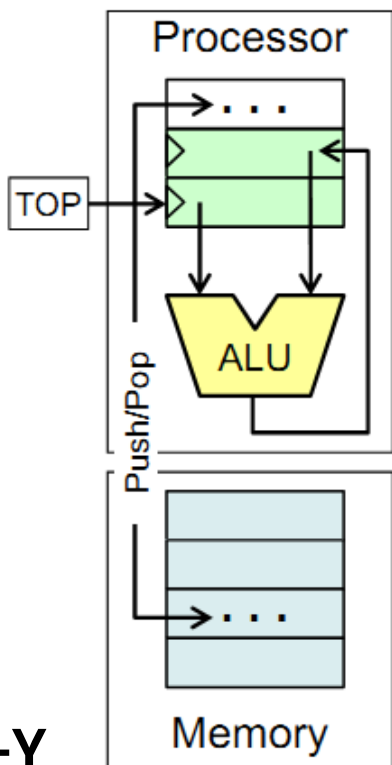
2.1 指令系统结构的分类

- ◆ 例：表达式 $Z=X+Y$ 在4种类型指令系统结构上的代码
- ◆ 假设：X、Y、Z均保存在存储器单元中，并且不能破坏X和Y的值。

堆栈型	累加器型	寄存器（RM型）	寄存器（RR型）
push X	load X	load R1, X	load R1, X
push Y	add Y	add R1, Y	load R2, Y
add	store Z	store R1, Z	add R3, R1, R2
pop Z			store R3, Z

ISA 的演进

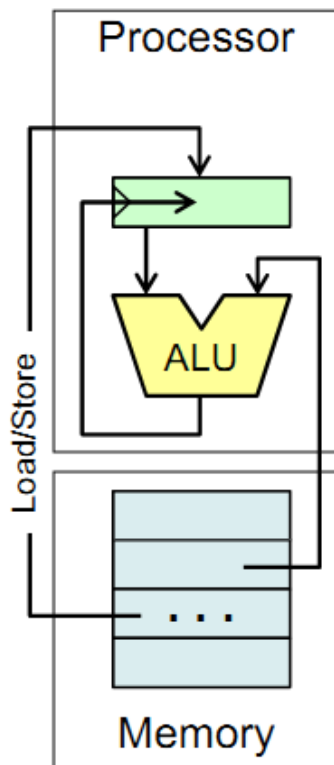
堆栈型



$Z=X+Y$

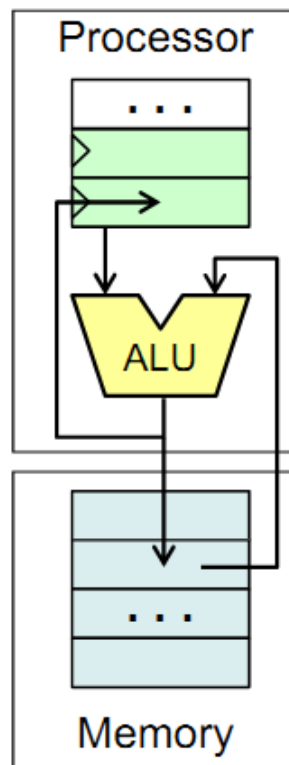
push X
push Y
add
pop Z

累加器型



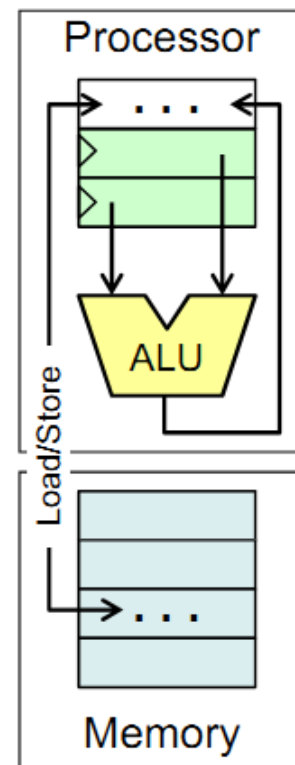
load X
add Y
store Z

寄存器RM型



load R1, X
add R1, Y
store R1, Z

寄存器RR型



load R1, X
load R2, Y
add R3, R1, R2
store R3, Z

通用寄存器结构

- ◆ 现代指令系统结构的主流
- ◆ 在灵活性和提高性能方面有明显的优势
 - 跟其他的CPU内部存储单元一样，寄存器的访问速度比存储器快。
 - 对编译器而言，能更加容易、有效地分配和使用寄存器。
 - 寄存器可以用来存放变量。
 - （1）减少对存储器的访问，加快程序的执行速度；
 - （2）用更少的地址位（相对于存储器地址来说）来对寄存器进行寻址，从而有效地减少程序的目标代码的大小。

指令集结构的分类

- 通用寄存器型 (General Purpose Register):

- Register-memory (RM型):

2 address `add A B` ; $EA[A] \leftarrow EA[A] + EA[B]$

3 address `add A B C` ; $EA[A] \leftarrow EA[B] + EA[C]$

- Load/Store (RR型):

3 address `add Ra Rb Rc` ; $Ra \leftarrow Rb + Rc$

`load Ra Rb` ; $Ra \leftarrow mem[Rb]$

`store Ra Rb` ; $mem[Rb] \leftarrow Ra$

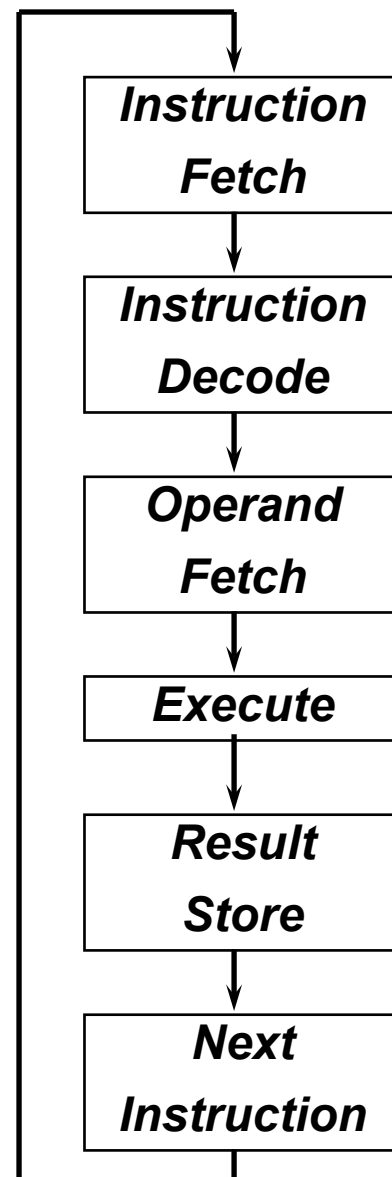
- 存储器-存储器型 (目前已经没有)

ALU指令中操作数个数 和存储器操作数个数的典型组合

ALU指令中存 储器操作数的个数	ALU指令中 操作数的最多个数	结构 类型	机器实例
0	3	RR	MIPS, SPARC, Alpha, PowerPC, ARM
1	2	RM	IBM 360/370, Intel 80x86, Motorola 68000
	3	RM	IBM 360/370
2	2	MM	VAX
3	3	MM	VAX

ISA: 必须说明哪些东西?

- ◆ 指令格式或编码方式。即如何编码?
- ◆ 操作数和操作结果的存放位置
 - 存放位置?
 - 多少个显式操作数?
 - 存储器操作数如何定位?
 - 哪些操作数可以或不可以放到存储器中?
- ◆ 数据类型和大小
- ◆ 寻址方式
- ◆ 支持哪些操作
- ◆ 下一条指令地址
 - jumps, conditions, branches
 - fetch-decode-execute is implicit!



2.2 寻址方式

- ◆ 80年以来几乎所有机器的存储器都是按**字节编址**
- ◆ 一个存储器地址可以访问：
 - 1个字节、2个字节、4个字节、更多字节
- ◆ 不同体系结构对**字**的定义是不同的
 - 16位字 (Intel X86) **32位字** (MIPS)
- ◆ 如何读32位字，两种方案
 - 每次一个字节，四次完成；每次一个字，一次完成
- ◆ 问题：
 - 如何将字节地址映射到字地址 (尾端问题)
 - 一个字是否可以存放在任何字节边界上 (对齐问题)
- ◆ 即尾端 (Endian)和对齐问题

尾端问题

- ◆ little endian, big endian, 在一个字内部的字节顺序问题

1. Little Endian byte ordering

x+3	x+2	x+1	x
Byte 3	Byte 2	Byte 1	Byte 0

 32-bit Register

✧ Memory address X = address of **least-significant** byte (Intel x86)

2. Big Endian byte ordering

x	x+1	x+2	x+3
Byte 0	Byte 1	Byte 2	Byte 3

 32-bit Register

✧ Memory address X = address of **most-significant** byte (SPARC)

- ◆ 如地址xxx00指定了一个字 (int)，存储器中从xxx00处连续存放AABBCCDD，则有两种方式：
 - Little endian 方式下xxx00位置是字的最低字节，整数值为DDCCBBAA，Intel 80x86，DEC Vax，DEC Alpha
 - Big endian 方式下xxx00位置是字的最高字节，整数值为AABBCCDD，IBM 360/370，Motorola 68k，MIPS，Sparc，HP PA

对齐问题

- ◆ 对s字节的对象访问地址为A，如果 $A \bmod s = 0$ 称为边界对齐
 - 边界对齐的原因是存储器本身读写的要求，存储器本身读写通常就是边界对齐的，对于不是边界对齐的对象的访问可能导致存储器的**两次访问**，然后再拼接出所需要的数。（或发生异常）

Address mod 8	0	1	2	3	4	5	6	7
Byte	Aligned	Aligned	Aligned	Aligned	Aligned	Aligned	Aligned	Aligned
2 Bytes	Aligned		Aligned		Aligned		Aligned	
2 Bytes		Misaligned		Misaligned		Misaligned		Misalign
4 Bytes	Aligned				Aligned			
4 Bytes		Misaligned				Misaligned		
4 Bytes			Misaligned				Misaligned	
4 Bytes				Misaligned				Misalign
8 Bytes	Aligned							
8 Bytes		Misaligned						

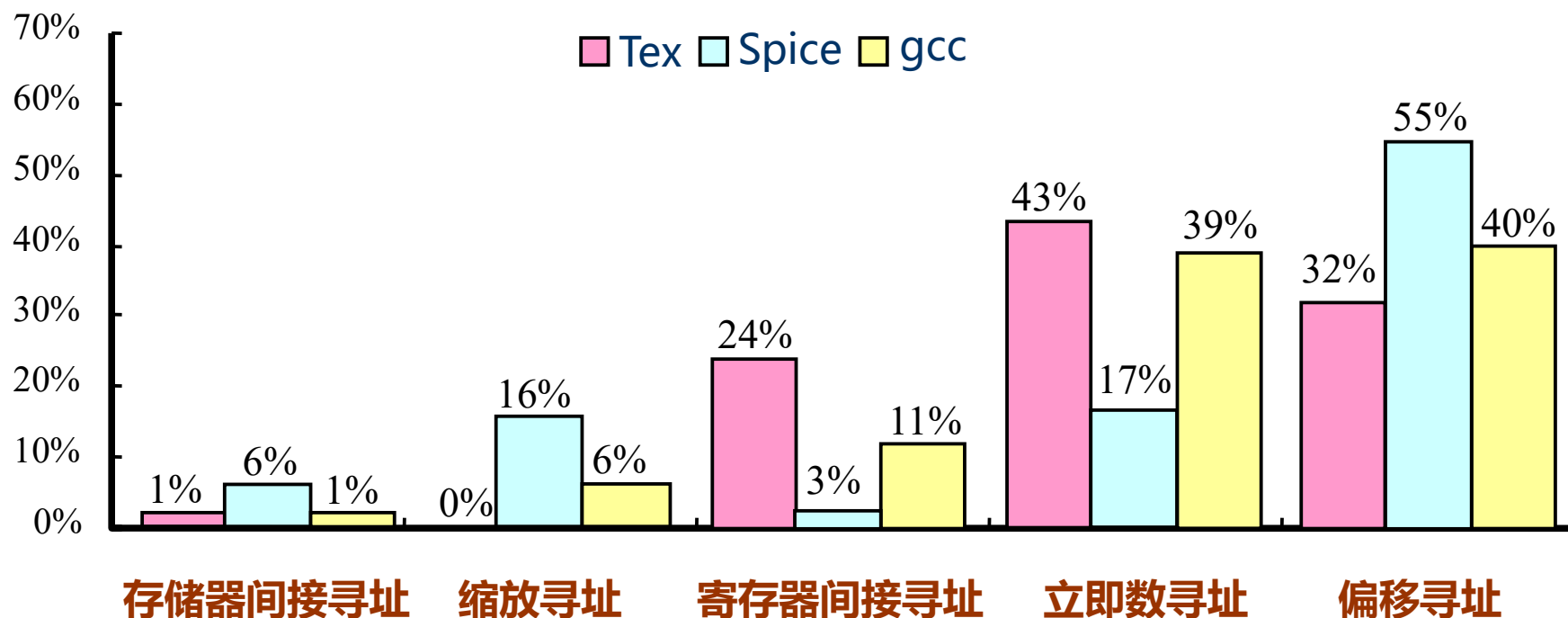
2.2 寻址方式

- ◆ 一种指令系统结构如何确定所要访问的数据的地址？
 - **寻址方式**：如何说明要访问的对象地址
 - 有效地址：由寻址方式说明的某一存储单元的实际存储器地址。有效地址 vs. 物理地址
- ◆ 当前的指令系统结构中所采用的一些操作数寻址方式
 - $\leftarrow$ ：赋值操作
 - Mem：存储器
 - Regs：寄存器组
 - 方括号：表示内容
 - Mem[]：存储器的内容
 - Regs[]：寄存器的内容
 - Mem[Regs[R1]]：以寄存器R1中的内容作为地址的存储器单元中的内容

寻址方式	指令实例	含 义
寄存器寻址	Add R4 , R3	Regs[R4]←Regs[R4] + Regs[R3]
立即值寻址	Add R4 , #3	Regs[R4]←Regs[R4] + 3
偏移寻址	Add R4 , 100(R1)	Regs[R4]←Regs[R4] + Mem[100+Regs[R1]]
寄存器间接寻址	Add R4 , (R1)	Regs[R4]←Regs[R4] + Mem[Regs[R1]]
索引寻址	Add R3, (R1 + R2)	Regs[R3]←Regs[R3] + Mem[Regs[R1]+Regs[R2]]
直接寻址或 绝对寻址	Add R1 , (1001)	Regs[R1]←Regs[R1] + Mem[1001]
存储器间接寻址	Add R1 , @(R3)	Regs[R1]←Regs[R1] + Mem[Mem[Regs[R3]]]
自增寻址	Add R1 , (R2)+	Regs[R1]←Regs[R1] + Mem[Regs[R2]] Regs[R2]←Regs[R2] + d
自减寻址	Add R1, -(R2)	Regs[R2]←Regs[R2] - d Regs[R1]←Regs[R1]+Mem[Regs[R2]]
缩放寻址	Add R1 , 100(R2)[R3]	Regs[R1]←Regs[R1] + Mem[100 + Regs[R2] + Regs[R3]*d]

各种寻址方式的使用情况统计结果

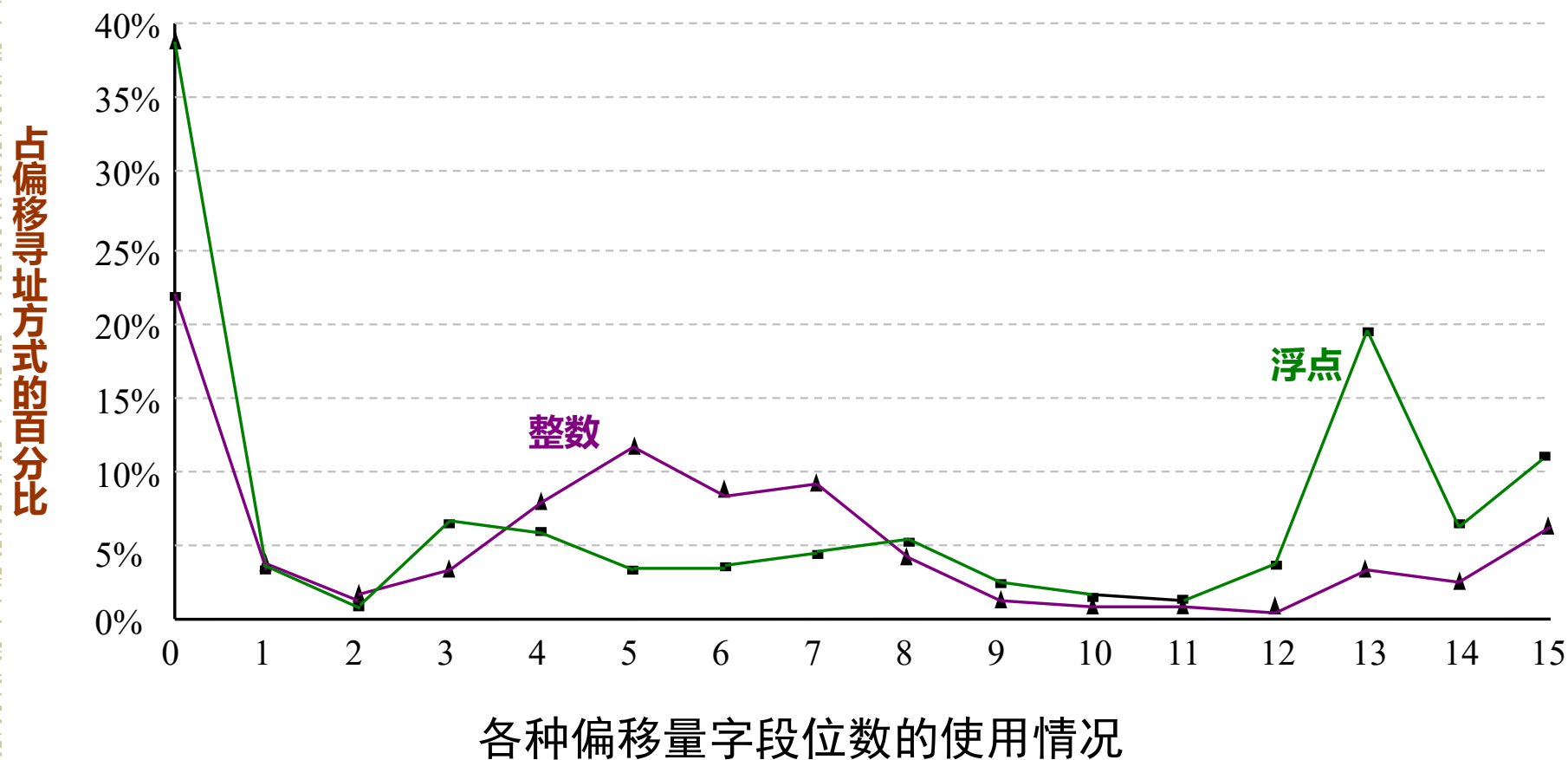
- 采用多种寻址方式可以显著地减少程序的指令条数，但可能增加计算机的实现复杂度以及指令的CPI。
- 在VAX机器上运行gcc、Spice和Tex 基准程序



- 立即数寻址方式和偏移寻址方式的使用频度最高。

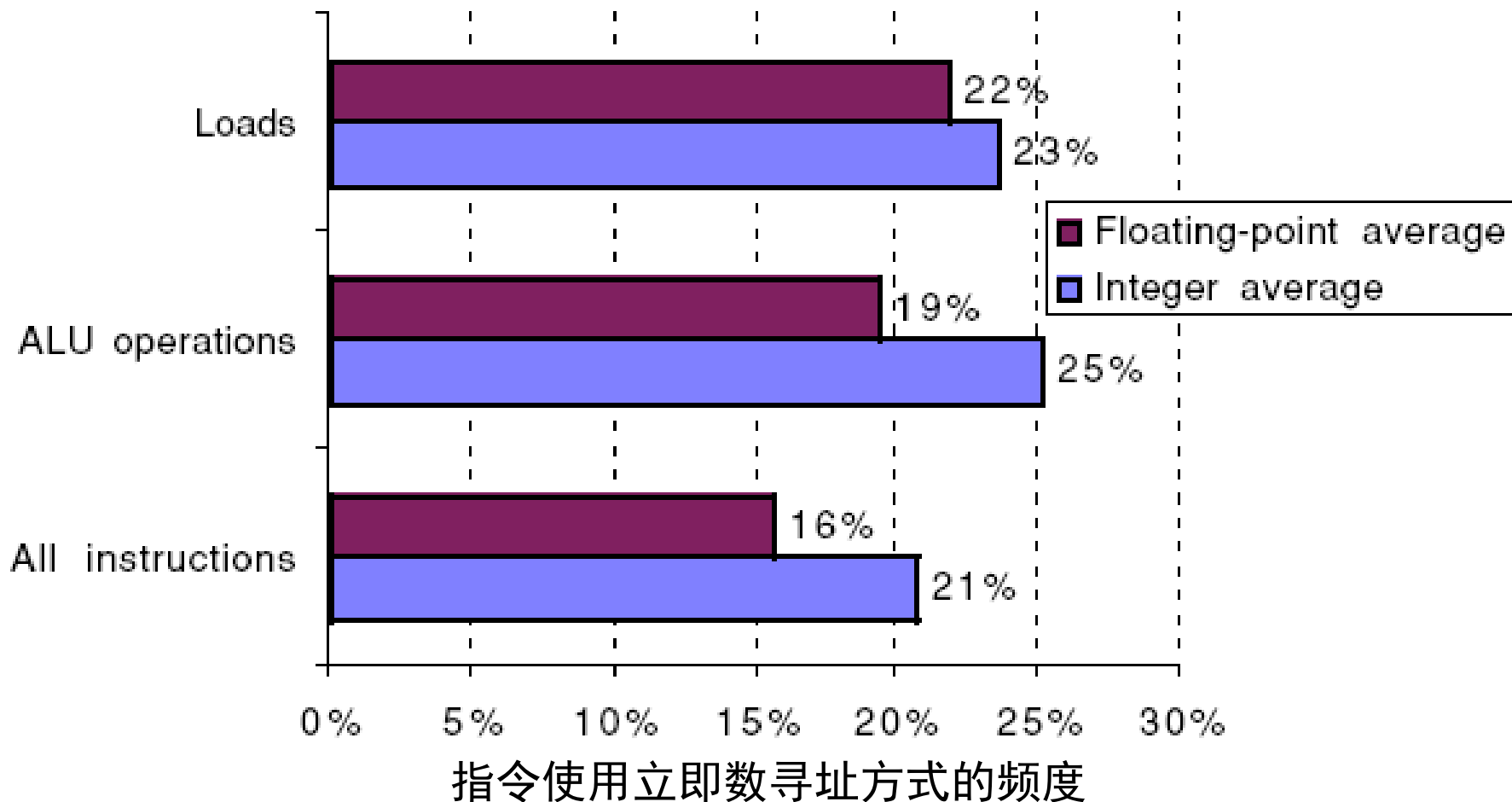
偏移量的取值范围

- 在load-store结构的机器（Alpha）上运行SPEC CPU2000 基准程序



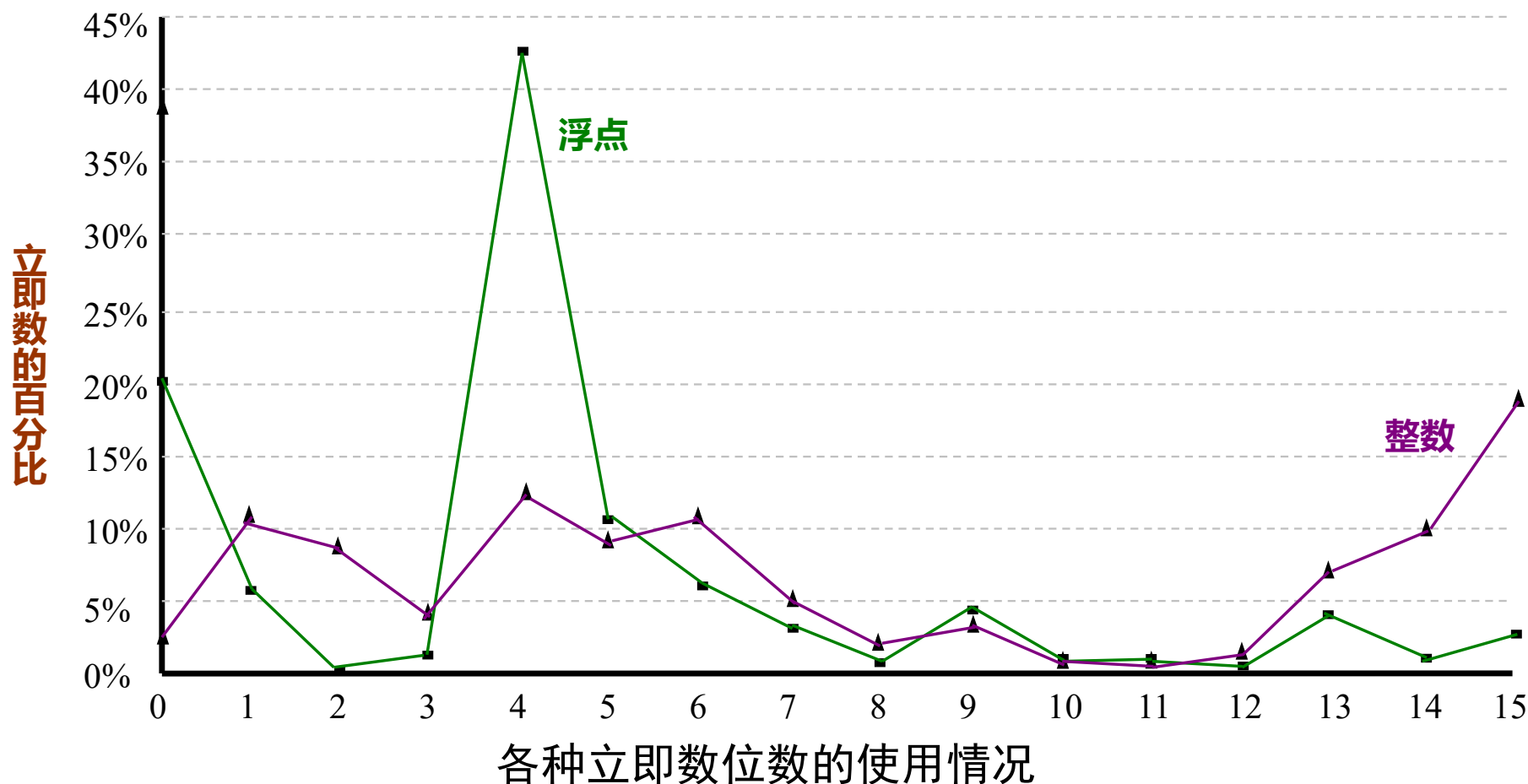
立即数寻址方式

- ◆ 在load-store结构的机器（Alpha）上运行SPEC CPU2000基准程序



立即数的取值范围

- ◆ 在load-store结构的机器（Alpha）上运行SPEC CPU2000 基准程序



寻址方式小结

- ◆ 重要的寻址方式：
 - 偏移寻址方式，立即数寻址方式，寄存器间接寻址方式
 - SPEC测试表明，使用频度达到 75%-99%
- ◆ 偏移字段的大小应该在 12 - 16 bits
 - 可满足75%-99%的需求
- ◆ 立即数字段的大小应该在 8 - 16 bits
 - 可满足50%-80%的需求
 - 最常用的是较小的立即数
 - 有时也会用到较大的立即数（主要是用于地址计算）
 - 在VAX机（支持32位立即数）上做过类似的统计，结果表明20%~25%的立即数超过16位。

2.3 指令系统的设计和优化

◆ 2.3.1 指令系统设计的基本原则

◆ 指令系统的设计

- 确定软、硬件功能分配，即确定哪些基本功能应该由硬件实现，哪些功能由软件实现比较合适。
 - 指令的**功能**设计
 - 指令**格式**的设计
- ◆ 在确定哪些基本功能用硬件来实现时，主要考虑3个因素：**速度**、**成本**、**灵活性**
 - **硬件实现的特点**
 - 速度快、成本高、灵活性差
 - 软件实现的特点
 - 速度慢、价格便宜、灵活性好

对指令系统的基本要求

◆ 基本要求：

- 完整性、规整性、正交性、高效率、兼容性

◆ 完整性：

- 在一个有限可用的存储空间内，对于任何可解的问题，编制计算程序时，指令集所提供的指令足够用。
- 要求指令集功能齐全、使用方便

◆ 兼容性：系列机，向后兼容

- 可以增加新指令，但不能删除和修改已有指令

◆ 高效率：指令执行速度快、使用频度高

◆ 正交性

- 数据类型独立于寻址方式
- 寻址方式独立于所要完成的操作

对指令集的基本要求

- ◆ **规整性**：让所有运算部件都能对称、均匀的在所有数据存储单元之间进行操作。对所有数据存储单元都能同等对待，无论是操作数或运算结果都可以无约束地存放任意数据存储单元中。具体表现为：
 - **对称性**：所有与指令集有关的存储单元的使用、操作码的设置等都是对称的。
 - 例如：在存储单元的使用上，所有通用寄存器都要同等对待。在操作码的设置上，如果设置了A-B的指令，就应该也设置B-A的指令
 - **均匀性**：指对于各种不同的操作数类型、字长、操作种类和数据存储单元，指令的设置都要同等对待。
 - 例如：如果某机器有5种数据表示，4种字长，两种存储单元，则要设置 $5 \times 4 \times 2 = 40$ 种同一操作的指令。

指令集结构中操作的分类

- ◆ 下表为许多指令集结构都包含的一些指令类型
 - 前4类属于通用计算机系统的基本指令
 - 对于最后4种类型的操作，不同指令集结构的支持大不相同

操作类型	实 例
算术和逻辑运算	算术运算和逻辑操作：加，减，乘，除，与，或等
数据传输	load, store
控制	分支，跳转，过程调用和返回，自陷等
系统	操作系统调用，虚拟存储器管理等
浮点	浮点操作：加，减，乘，除，比较等
十进制	十进制加，十进制乘，十进制到字符的转换等
字符串	字符串移动，字符串比较，字符串搜索等
图形	像素操作，压缩/解压操作等

ISA对操作类型的选择

- ◆ 在设计指令集结构时，有两种截然不同的设计策略
 - 产生了两类不同的计算机系统
- ◆ **CISC**（复杂指令集计算机）
 - 增强指令功能，把越来越多的功能交由硬件来实现，并且指令的数量也是越来越多。
- ◆ **RISC**（精简指令集计算机）
 - 尽可能地把指令集简化，不仅指令的条数少，而且指令的功能也比较简单。

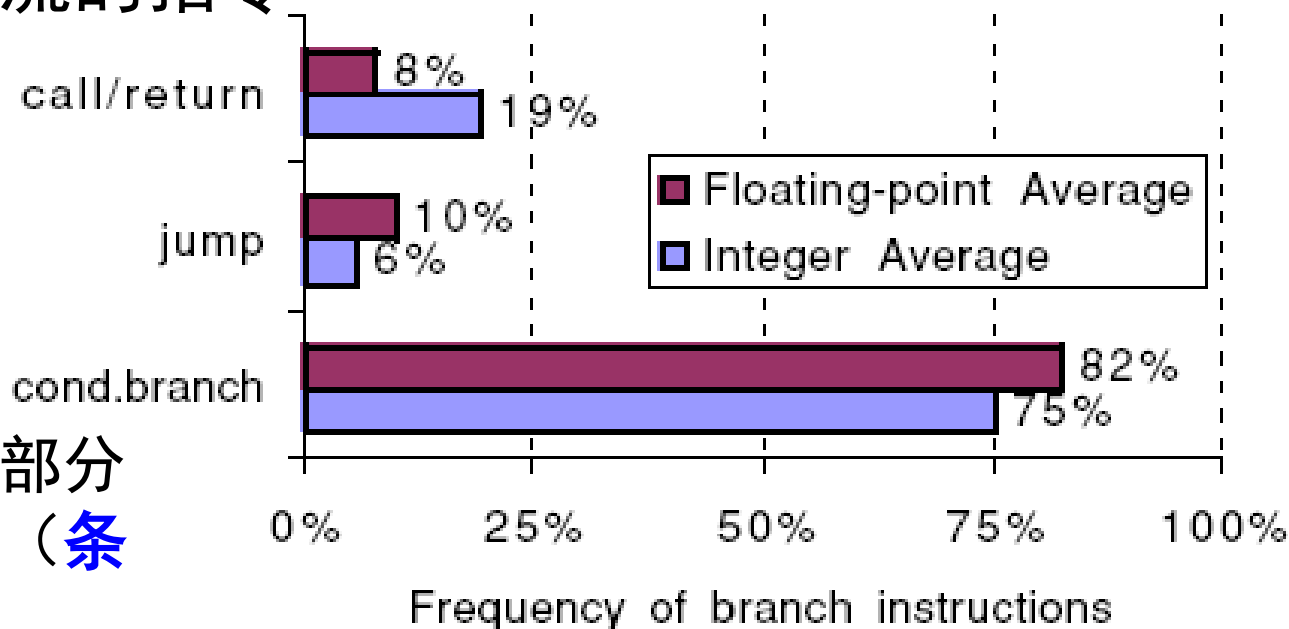
2.3.2 控制指令

- ◆ 控制指令是用来改变控制流的。
 - **跳转**：当指令是无条件改变控制流时，称之为跳转指令。
 - **分支**：当控制指令是有条件改变控制流时，则称之为分支指令。

- ◆ 能够改变控制流的指令

- 跳转
- 分支
- 过程调用
- 过程返回

- ◆ 改变控制流的大部分指令是**分支指令**（**条件转移**）



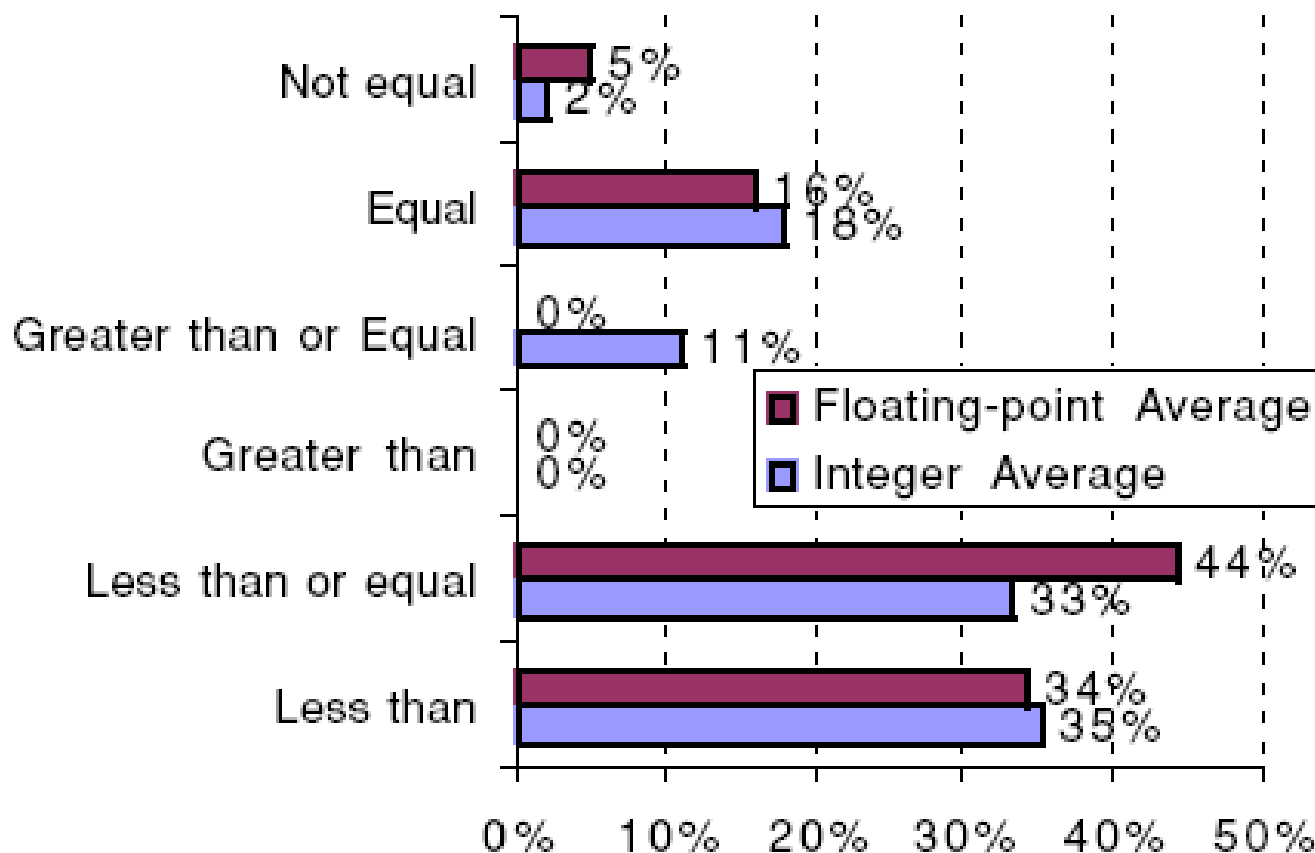
表示分支条件的3种方法

- 分支条件的方法及其优缺点

名称	检测分支条件的方法	优点	缺点
条件码 (CC)	检测由ALU操作设置的一些特殊的位 (即CC)	可以自由设置分支条件	条件码是增设的状态。而且它限制了指令的执行顺序, 因为要保证条件码能顺利地传送给分支指令
条件寄存器	比较指令把比较结果放入任何一个寄存器, 检测时就检测该寄存器	简单	占用了一个寄存器
比较与分支	比较操作是分支指令的一部分, 通常这种比较是受到一定限制的	用一条指令 (而不是两条) 就能实现分支	当采用流水方式时, 该指令的操作可能太多, 在一拍内做不完

分支比较类型比较

- Alpha Architecture with full optimization for Spec CPU2000, showing the average of integer programs(CINT2000) and the average of floating-point programs (CFP2000)



转移目标地址的表示

◆ PC相对寻址

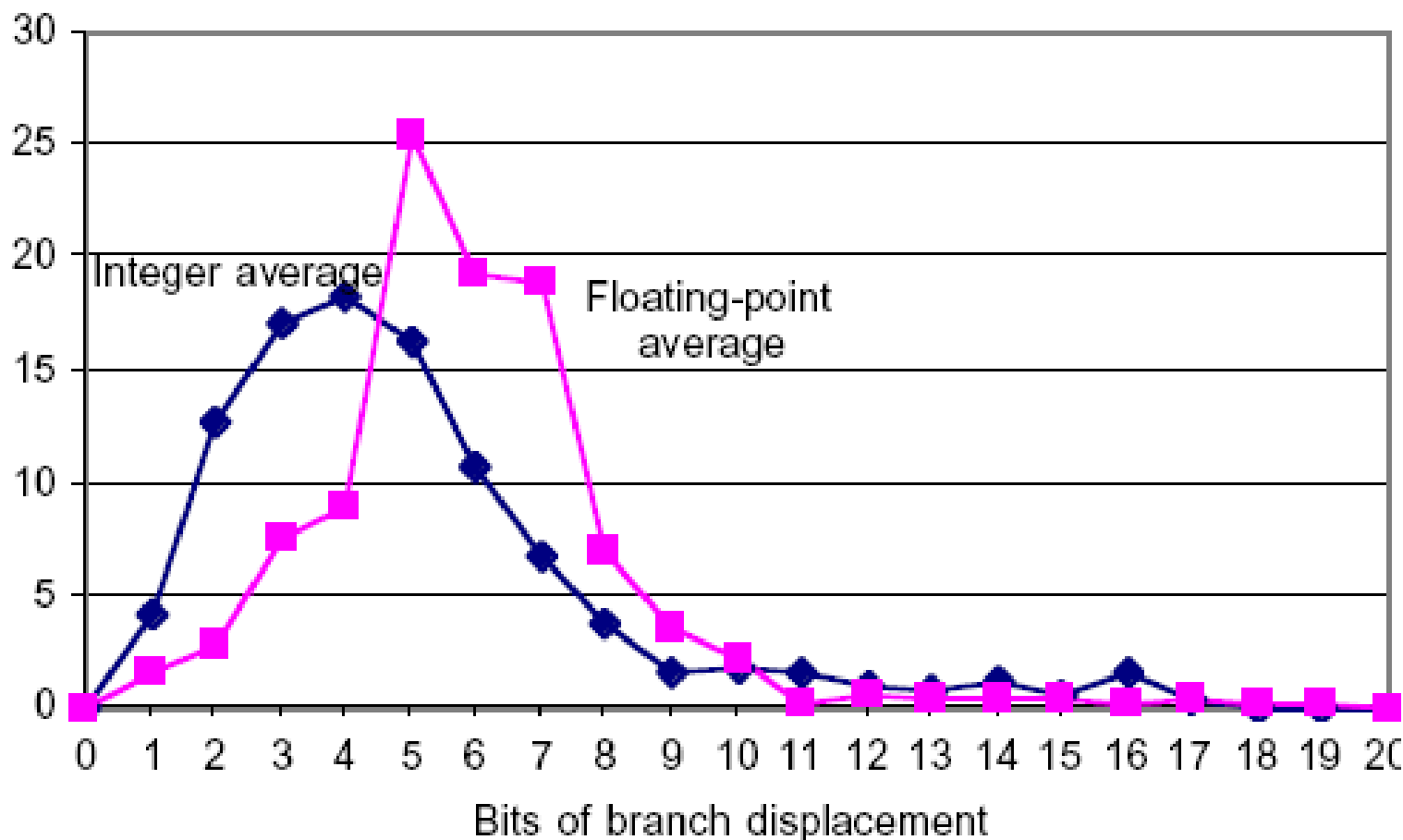
- 在指令中提供一个偏移量，由该偏移量和程序计数器(PC)的值相加而得出目标地址。

◆ 动态的转移地址方式(通过寄存器间接跳转)：

- 编译时不知道目标地址，程序执行时动态确定
 - Case or switch statements
 - Virtual function or methods
 - High-order functions or function pointers
 - Dynamically shared libraries
- 转移地址放到某一寄存器中，通过寄存器间接跳转

- ## ◆ 优点
- 有效地减少表示该目标地址所需要的位数
 - 位置无关（代码可被装载到主存的任意位置执行）

转移目标地址与当前指令的距离



◆ 关键：确定偏移量字段的长度

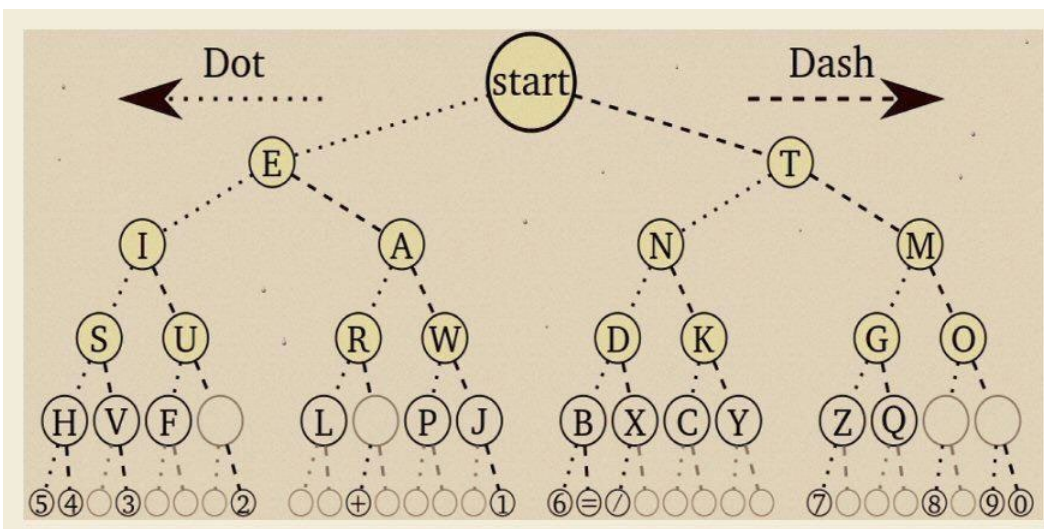
- 模拟结果表明：采用4~8位的偏移量字段（以指令字为单位）就能表示大多数控制指令的转移目标地址了。

2.3.3 指令操作码的优化

- ◆ 指令由两部分组成：操作码、地址码
- ◆ 指令格式的设计
 - 确定指令字的编码方式，包括操作码字段和地址码字段的编码和表示方式。
- ◆ 问题提出的背景和目标
 - 如何节省程序的存贮空间；
 - 指令格式怎样规整，尽可能减少译码时间和取指令的时间
- ◆ 问题：空间 vs 时间？
- ◆ 指令格式的优化：
 - 如何用最短的位数来表示指令的操作信息和地址信息

摩尔斯电码

- ◆ 摩尔斯电码也被称作摩斯密码，是一种时通时断的信号代码，通过不同的排列顺序来表达不同的英文字母、数字和标点符号。它发明于1837年，是一种早期的数字化通信形式。
- ◆ 各国就在1999年停止了对摩尔斯码的使用。



A ● —
B — ● ● ●
C — ● — ●
D — ● ●
E ●
F ● ● — ●
G — — ●
H ● ● ● ●
I ● ●
J ● — — —
K — ● —
L ● — ● ●
M — —
N — ●
O — — —
P — — — ●
Q — — — —
R ● — ●
S ● ● ●
T —

U ● ● —
V ● ● ● —
W ● — —
X — ● ● —
Y — — — —
Z — — — ● ●

1 ● — — — —
2 ● ● — — —
3 ● ● ● — —
4 ● ● ● ● —
5 ● ● ● ● ●
6 — ● ● ● ●
7 — — ● ● ●
8 — — — ● ●
9 — — — — ●
0 — — — — —

哈(赫)夫曼编码

◆ 基本思想

- 当各种事件发生的概率不均等时，可以对发生概率最高的事件用最短的位数（时间）来表示（处理），而对于出现概率较低的事件，则可以用较长的位数（时间）来表示（处理），从而使总的平均位数（时间）缩短。

◆ 构造哈夫曼树的方法

- 将各事件按其使用频度从小到大依次排列；
- 每次从中选择两个频度值最小的结点，将其合并成一个新的结点，并把新结点画在所选结点的上面，
- 然后用两条边把新结点分别与那两个结点相连。新结点的频度值是所选两个结点的频度值的和。
- 把新结点与其他剩余未结合的结点一起，再以上面的步骤进行处理，反复进行，直到全部结点都结合完毕、形成根结点为止。

哈夫曼编码

◆ 操作码优化的程度可以用**信息熵**来衡量

- 表示用二进制编码表示n个码点时，理论上的最短平均编码长度。

$$H = - \sum_{i=1}^n p_i \log_2 p_i$$

◆ 例2.1 假设某模型机有7条指令，这些指令的使用频率如下表所示。

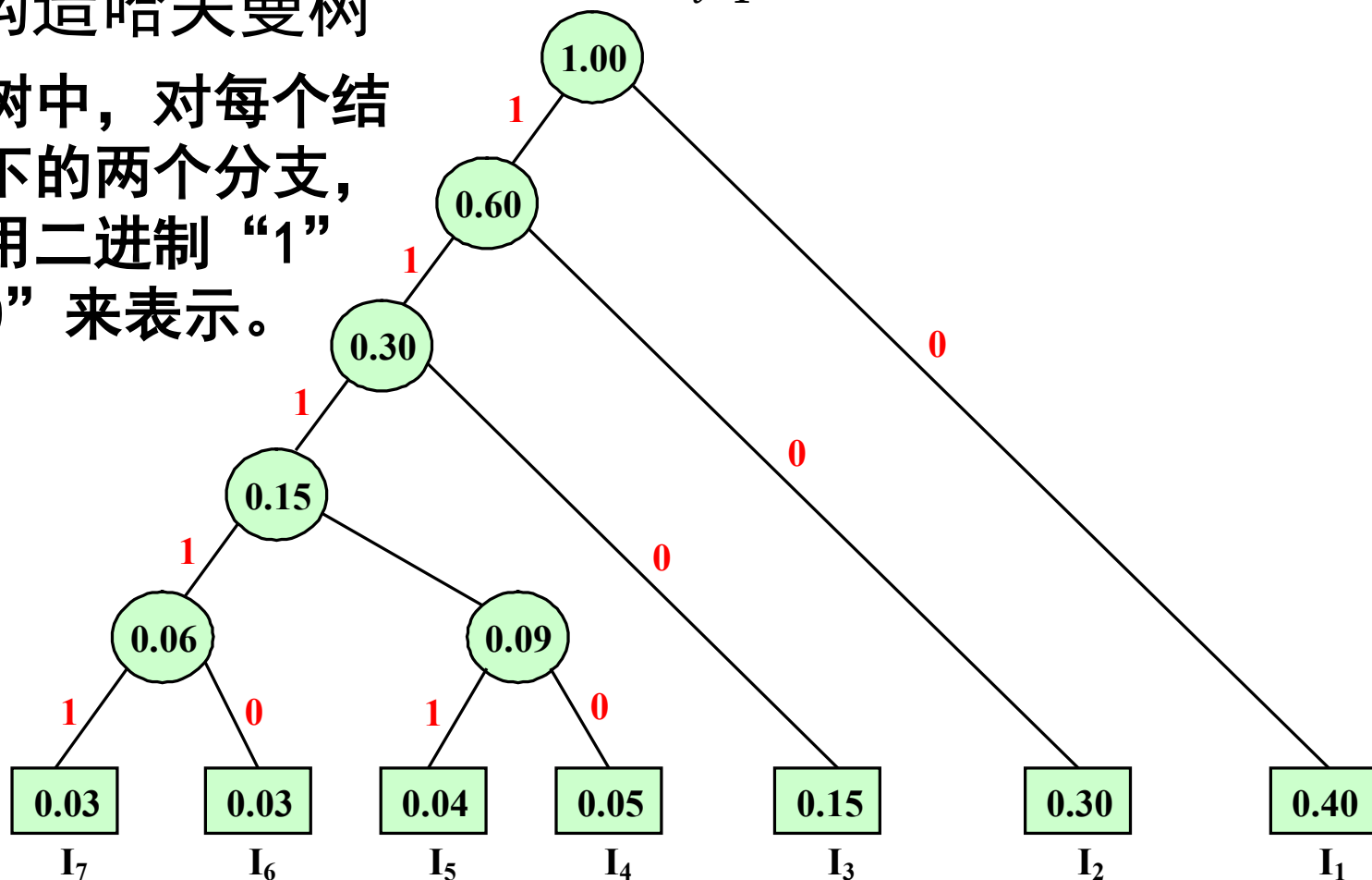
- (1) 计算这7条指令的操作码编码的最短平均码长；
- (2) 画出哈夫曼树，写出这7条指令的哈夫曼编码，并计算该编码的平均码长和信息冗余量。

指令	I ₁	I ₂	I ₃	I ₄	I ₅	I ₆	I ₇
频率P _i	0.40	0.30	0.15	0.05	0.04	0.03	0.03

哈夫曼编码举例

- ◆ 解 (1) 最短平均码长 $H = - \sum_{i=1}^7 p_i \log_2 p_i = 2.17$
- ◆ (2) 构造哈夫曼树

- 在该树中，对每个结点向下的两个分支，分别用二进制“1”和“0”来表示。



每个叶结点
分别对应于
一条指令。

哈夫曼编码举例

- ◆ 从哈夫曼树得到每个指令的哈夫曼编码
 - 对于任意一条指令 I_i ($i=1, 2, \dots, 7$)，从哈夫曼树根结点出发、沿一条路径连接到叶结点 I_i ，把途中所经过的各分支的“0”和“1”按从左到右的顺序记录下来，便是该指令的哈夫曼编码。

指令	I_1	I_2	I_3	I_4	I_5	I_6	I_7
频率 P_i	0.40	0.30	0.15	0.05	0.04	0.03	0.03
编码	0	10	110	11100	11101	11110	11111
长度 L_i	1	2	3	5	5	5	5

- 该哈夫曼编码的平均码长是：

$$L = \sum_{i=1}^7 p_i l_i = 2.20$$

- 信息冗余量为 $\frac{2.20 - 2.17}{2.20} \approx 1.36\%$

优缺点

哈夫曼编码举例

◆ 扩展操作码

平均码长为2.3，比哈夫曼的2.2大一点

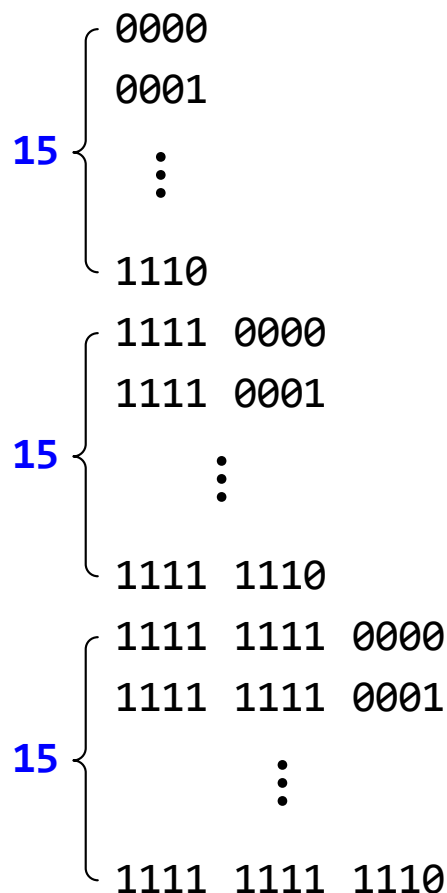
- 位于定长二进制编码和哈夫曼编码之间的一种编码方案。
- 采用有限几种固定长度的码长，仍然**采用高概率的用短码、低概率用长码**的哈夫曼压缩思想，使操作码平均长度缩短。
- 采用2-4的扩展操作码，可以得到如下表所示的编码方案。

指令	I ₁	I ₂	I ₃	I ₄	I ₅	I ₆	I ₇
频率P _i	0.40	0.30	0.15	0.05	0.04	0.03	0.03
编码	0	10	110	11100	11101	11110	11111
长度L _i	1	2	3	5	5	5	5
扩展编码	00	01	10	1100	1101	1110	1111
长度L _i	2	2	2	4	4	4	4

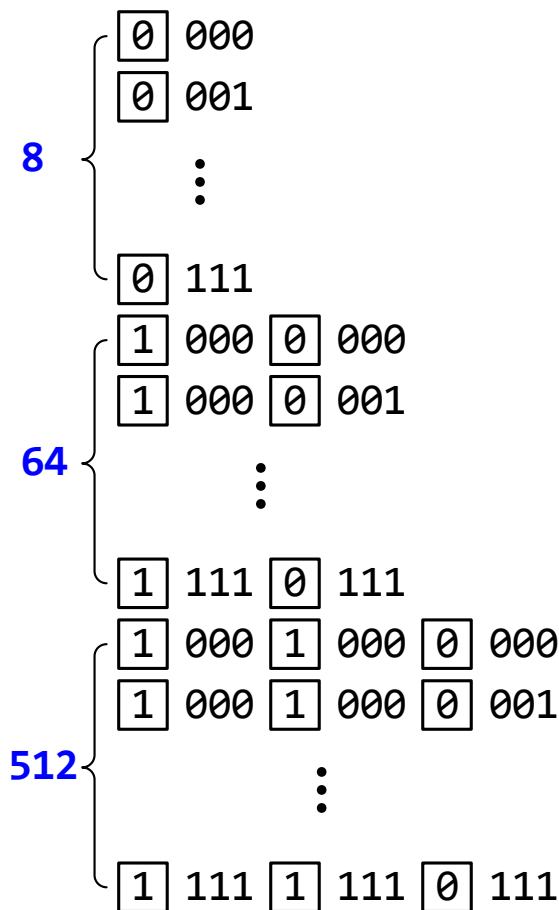
等长扩展码

- ◆ 为了便于分级译码，在早期的计算机上，一般采用等长扩展码
- ◆ 例如：15/15/15法和8/64/512法

- ◆ 选用哪种编码法取决于指令使用频度 p_i 的分布。
- ◆ 衡量标准：看哪种编码法能使平均码长最短。



15/15/15 编码法



8/64/512 编码法

定长操作码

- ◆ 固定长度的操作码：所有指令的操作码都是同一的长度（如8位）。
- ◆ 许多计算机都采用（特别是RISC结构的计算机）
- ◆ 保证操作码的译码速度、减少译码的复杂度。
- ◆ 以程序的存储空间为代价来换取硬件实现上的好处

2.3.4 指令字格式的优化

◆ 两种表示寻址方式的方法

- 将**寻址方式编码于操作码**中，由操作码描述相应操作的寻址方式。
 - 适合：处理机采用load-store结构，寻址方式只有很少几种。
- **设置专门的地址描述符**，由地址描述符表示相应操作数的寻址方式。
 - 适合：处理机具有多种寻址方式，且指令有多个操作数。

◆ 考虑因素

- 机器中寄存器的个数和寻址方式的数目对指令平均字长的影响以及它们对目标代码大小的影响。
- 所设计的指令格式便于硬件处理，特别是流水实现。
- 指令字长应该是字节（8位）的整数倍，而不能是随意的位数。

2.3.4 指令字格式的优化

◆ 指令集的3种编码格式

- **变长编码格式、定长编码格式、混合型编码格式**
- 如果代码长度最重要，那么使用变长指令格式
- 如果性能至关重要，使用固定长度指令

◆ 变长编码格式

- 当指令集的寻址方式和操作种类很多时，这种编码格式是最好的。
- 用最少的二进制位来表示目标代码。
- 可能会使各条指令的字长和执行时间相差很大。

操作码	地址描述符 1	地址码 1	...	地址描述符 n	地址码 n
-----	---------	-------	-----	---------	-------

2.3.4 指令字格式的优化

◆ 定长编码格式

- 将操作类型和寻址方式一起编码到操作码中。
- 当寻址方式和操作类型非常少时，这种编码格式非常好。
- 可以有效地降低译码的复杂度，提高译码的速度。
- **大部分RISC的指令集均采用这种编码格式。**

操作码	地址码 1	地址码 2	地址码 3
-----	-------	-------	-------

◆ 混合型编码格式

- 以期达到既能够减少目标代码长度又能降低译码复杂度的目标。
- 提供若干种固定的指令字长。

操作码	地址描述符	地址码
-----	-------	-----

操作码	地址描述符 1	地址描述符 2	地址码
-----	---------	---------	-----

操作码	地址描述符 1	地址码 1	地址码 2
-----	---------	-------	-------

2.4 指令系统的发展和改进

◆ 2.4.1 沿CISC方向发展和改进指令系统

- CISC指令系统的特点：指令数量多、功能多样
- CISC结构追求的目标：**强化指令功能，减少程序的指令条数，以达到提高性能的目的。**

◆ 增强指令功能主要是从以下几个方面着手：

- 1、面向目标程序的优化是提高计算机系统性能最直接方法

■ 优化目标

- ◆ 缩短程序的长度
- ◆ 缩短程序的执行时间

■ 优化方法

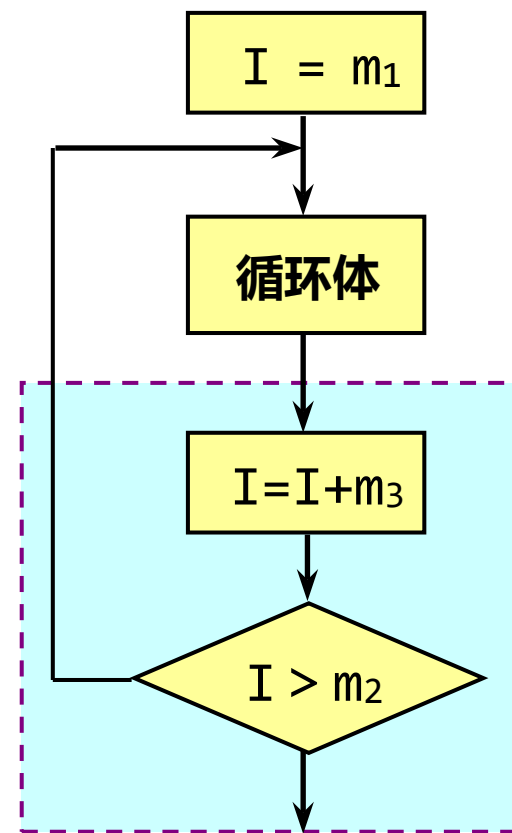
- ◆ 统计分析目标程序执行情况，找出使用频度高，执行时间长的指令或指令串
- ◆ 优化使用频度高的指令
- ◆ 用新的指令代替使用频度高的指令串

优化目标程序的主要途径

- ◆ 1) 增强运算型指令的功能
 - 如 $\sin(x)$, $\cos(x)$, $\text{SQRT}(X)$, 甚至多项式计算
 - 如用一条三地址指令完成
 - $P(X) = C(0) + C(1)X + C(2)X^2 + C(3)X^3 + \dots$
- ◆ 2) 增强数据传送类指令的功能
 - 主要是指数据块传送指令
 - R-R, R-M, M-M之间的数据块传送可有效的支持向量和矩阵运算, 如IBM370
 - R-Stack之间设置数据块传送指令, 能够在程序调用和程序中断时, 快速保存和恢复程序现场, 如 VAX-11
- ◆ 3) 增强程序控制指令的功能
 - 在CISC中, 一般均设置了多种程序控制指令, 正常仅需要转移指令和子程序控制指令

增强程序控制指令的功能-举例

- ◆ 例如：循环在程序中占有相当大的比例，所以在指令上提供专门的支持。
- ◆ 循环控制部分通常用3条指令完成：
 - 一条加法指令，一条比较指令，一条分支指令
 - 统计结果表明，控制循环指令在整个循环程序中占据了很大的比例
- ◆ **设置循环控制指令**，用一条指令完成上述3条指令的功能。
 - IBM 370，大于转移指令
- ◆ 仅对频繁使用的子程序或指令串，用较强功能的指令取而代之才划算。



一般循环程序的结构

CISC计算机ISA的功能设计

- ◆ 2、面向高级语言和编译程序改进指令系统
 - 优化目标：主要是缩小高级语言与机器语言的语义差距
- ◆ 优化方法：
 - 1) 增强面向HL和Compiler支持的指令功能
 - 统计分析源程序中各种语句的使用频度和执行时间
 - 增强相关指令的功能，优化使用频度高、执行时间长语句
 - 增加专门指令，以缩短目标程序长度，减少目标程序执行时间，缩短编译时间
 - 2) 高级语言计算机系统
 - 缩小HL和ML的差别时，走到极端，就是把HL和ML合二为一，即所谓的高级语言计算机。在这种机器中，高级语言不需要经过编译，直接由机器硬件来执行。如LISP机，PROLOG机

主要语句的使用频度-举例

◆ FORTRAN语言和COBOL语言中各种主要语句使用频度

语言	一元赋值	其他赋值	IF	GOTO	I/O	DO	CALL	其他
FORTRAN	31.0	15.0	11.5	10.5	6.5	4.5	6.0	15.0
COBOL	42.1	7.5	19.1	19.1	8.46	0.17	0.17	3.4

◆ 观察结果：

- 1) 一元赋值在其中比例最大，增强数据传送类指令功能，缩短这类指令的执行时间是对高级语言非常有力的支持
- 2) 其他赋值语句中，增1操作比例较大，许多机器都有专门的增1指令
- 3) 条件转移和无条件转移占22%，38.2%，因此增强转移指令的功能，增加转移指令的种类是必要的

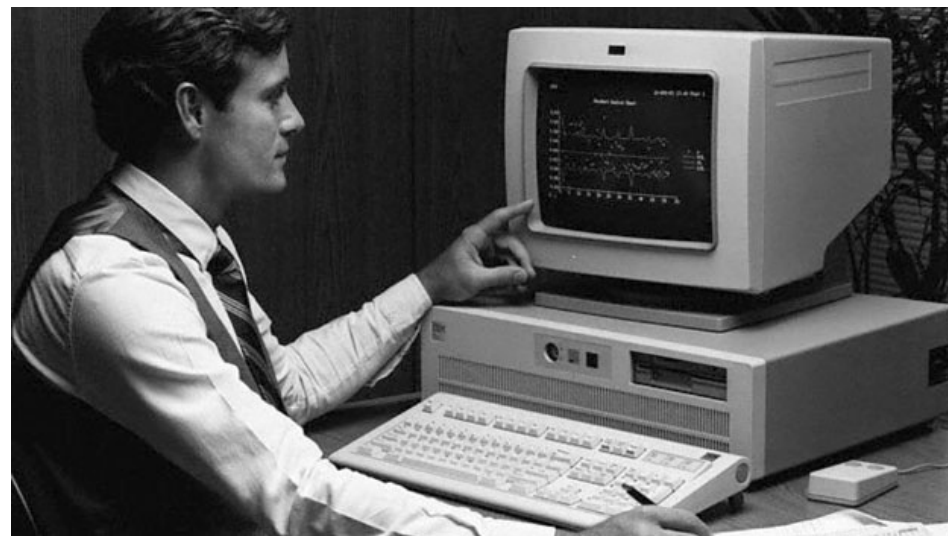
CISC计算机ISA的功能设计

◆ 3、面向操作系统的优化实现改进指令集

- 操作系统和计算机系统结构是紧密联系的，任何一种计算机系统通常需要操作系统，而操作系统又必须用指令系统来实现，操作系统的实现在很大程度上取决于系统结构的支持。
- 指令集对操作系统的支持主要有：
 - 处理机工作状态和访问方式的切换。
 - 进程的管理和切换。
 - 存储管理和信息保护。
 - 进程的同步与互斥，信号灯的管理等。
- 支持操作系统的有些指令属于特权指令，一般用户程序是不能使用的。

2.4.2 沿RISC方向发展和改进指令系统

- ◆ 1974 John Cocke 和他的团队在给电话交换机设计控制器时，开始**研究指令系统的合理性问题**，John Cocke（1987年图灵奖获得者）70年代末IBM推出 801CPU，1986年推出 IBM RT PC（或称6150）



- ◆ IBM RT PC
- ◆ The RISC Technology Personal Computer (RT PC) was introduced in 1986, and featured the 32-bit RISC architecture.

CISC指令系统结构存在的问题

- ◆ 1979 David Patterson 研究了CISC指令系统主要存在以下几方面的问题：
 - 指令使用频度，二八定律；
 - 指令集庞大，使得VLSI设计困难，不利于单片集成；CISC指令系统复杂 → 增加研制时间和成本，容易出错；
 - 许多复杂指令操作复杂，CPI较大，运行速度慢；
 - 各条指令不规整，不利于先进计算机体系结构技术来提高系统的性能。

80x86最常用的10条指令

排序	80x86指令	指令执行频度（占比）
1	load	22%
2	条件分支	20%
3	比较	16%
4	store	12%
5	add	8%
6	and	6%
7	sub	5%
8	寄存器-寄存器间数据移动	4%
9	call	1%
10	return	1%
合计：		95%

设计RISC机器遵循的原则

- ◆ **指令条数少而简单**。只选取使用频度很高的指令，在此基础上补充一些最有用的指令。
- ◆ **采用简单而又统一的指令格式**，并减少寻址方式；指令字长都为32位或64位。
- ◆ 指令的执行在**单个机器周期内**完成。
 - 采用流水线机制
- ◆ 只有load和store指令才能访问存储器，其他指令的操作都是在寄存器之间进行。
 - 即采用load-store结构
- ◆ 大多数指令都采用硬连逻辑来实现。
- ◆ 强调优化编译器的作用，为高级语言程序生成优化的代码。
- ◆ 充分利用流水技术来提高性能。

早期的RISC微处理器

- ◆ 1981年，Berkeley分校的Patterson等人的32位微处理器RISC I：
 - 31条指令，指令字长都是32位，78个通用寄存器，时钟频率为8 MHz，性能比MC68000和Z8000快3~4倍。
- ◆ 1983年的RISC II：
 - 指令条数为39，通用寄存器个数为**138**，时钟频率为12 MHz。
 - 后来发展成了Sun公司的SPARC系列微处理器。
- ◆ 1981年，Stanford大学Hennessy等人的MIPS
 - 后来发展成了MIPS Rxxx系列微处理器。
- ◆ IBM的801
 - IBM RT-PC
 - RS6000
- ◆ **共同特点：**
 - 采用load-store结构
 - 指令字长为32位
 - 采用高效的流水技术

RISC的定义和特点

- ◆ **RISC是一种计算机体系结构的设计思想**，它不是一种产品。RISC是近代计算机体系结构发展史中的一个里程碑，直到现在，RISC没有一个确切的定义
- ◆ 早期对RISC特点的描述
 - 大多数指令在单周期内完成
 - 采用Load/Store结构
 - 硬布线控制逻辑
 - 减少指令和寻址方式的种类
 - 固定的指令格式
 - 注重代码的优化
- ◆ 从目前看，RISC体系结构还应具有如下特点：
 - 面向寄存器结构
 - 十分重视流水线的执行效率—尽量减少断流
 - 重视优化编译技术
- ◆ **减少指令平均执行周期数是RISC思想的精华**

问题

- ◆ RISC的指令系统精简了，CISC中的一条指令可能在RISC中由一串指令才能完成，那么为什么RISC执行程序的速度比CISC还要快？
- ◆ **CPU总时间 = IC X CPI X T**

	IC	CPI	T
CISC	1	2~15	33ns~5ns
RISC	1.3~1.4	1.1~1.4	10ns~2ns

- ◆ IC：实际统计结果，RISC的IC只比CISC长30%~40%
- ◆ CPI：CISC CPI一般在4~6之间，RISC一般CPI=1，Load/Store为2
- ◆ T：RISC采用硬布线逻辑，指令要完成的功能比较简单

减少CPI 的主要途径

◆ 硬件方面：

- 硬布线控制逻辑
- 减少指令和寻址方式的种类
- 使用固定格式
- 采用Load/Store
- 指令执行过程中设置多级流水线

◆ 软件方面：十分强调优化编译的作用

- ◆ 目前，超标量、超流水线处理机的CPI已经达到0.5，实际上用**IPC** (Instruction Per Cycle)更确切。

- Intel公司的80x86处理机的CPI在不断缩小
 - 8088的CPI大于20
 - 80286的CPI大约是5.5
 - 80386的CPI进一步减小到4左右
 - 80486的CPI已经接近2
 - Pentium处理机的CPI已经与RISC十分接近

CISC与RISC的主要特征对比

功能	CISC	RISC
指令系统	复杂、庞大	简单、精确
指令条数	>200	<100
指令格式	>4	<4
寻址方式	>4	<4
指令字长	不固定	32bit
可访存指令	不加限制	只有LOAD/STORE
各种指令使用频率	相差太大	相差不大
各种指令执行时间	相差太大	绝大多数一周期完成
优化编译系统	很难	较容易
程序源代码长度	短	长
控制逻辑实现方式	绝大多数 微程序 控制	绝大多数为 硬连线 控制

RISC的关键技术

◆ 延时转移技术

- 为了尽量保证**流水线**的执行效率，在转移指令之后插入一条有效的指令，而转移指令好像被延时了

◆ 指令取消技术

- 向后转移，向前转移，根据是否转移成功，取消下一条指令

◆ 重叠寄存器窗口技术

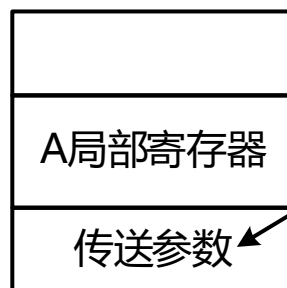
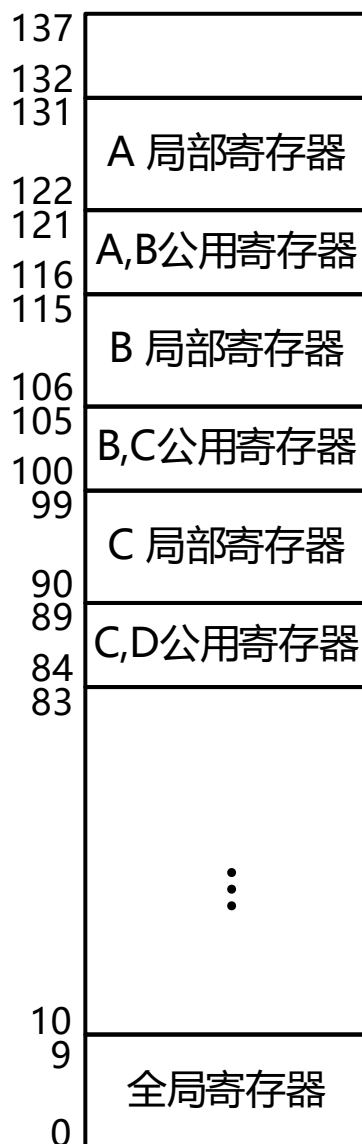
- 处理器中设置较多的寄存器堆，并划分很多窗口
- 每个过程使用其中相邻的三个窗口和一个公共的窗口
- 一个与前一个过程共用，一个与下一个过程共用，一个作为局部寄存器

◆ 指令流调整技术

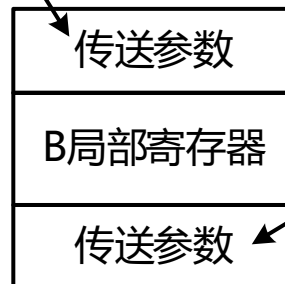
- 为了保持指令流水线高效率，不断流，优化编译器必须分析程序的数据流和控制流

◆ 硬件为主固件为辅

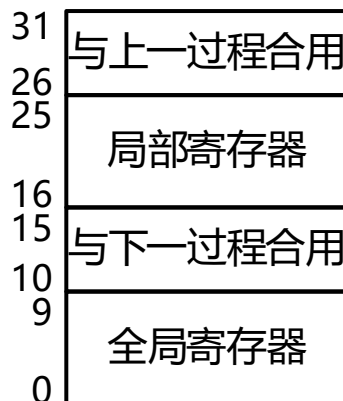
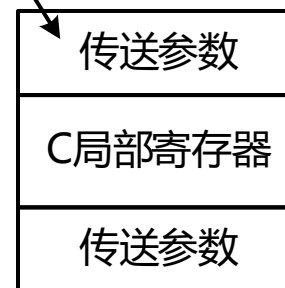
重叠寄存器窗口技术 (RISC II)



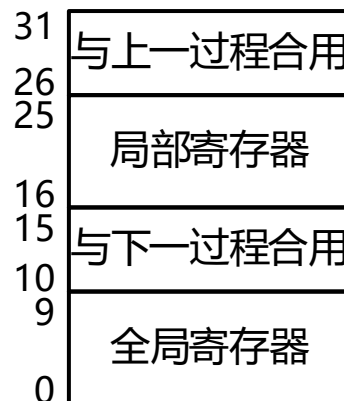
寄存器重叠



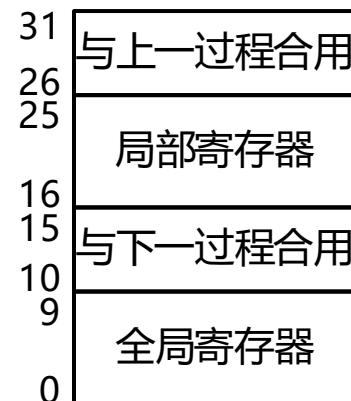
寄存器重叠



A过程的
寄存器窗口



B过程的
寄存器窗口



C过程的
寄存器窗口

重叠寄存器窗口技术

◆ 寄存器窗口技术：

- 把整个寄存器组分成很多小组，每个过程分配一个寄存器小组，当发生过程调用时，自动地把CPU转换到不同的寄存器小组使用，**不再需要作保存和恢复的操作**，这个寄存器小组就叫做寄存器窗口，相邻的寄存器窗口间有部分重叠的，**便于调用参数传送**。
- ◆ 采用重叠寄存器窗口技术使RISC在过程调用的执行时间、执行指令条数、访存次数等指标方面大大优于CISC处理机。
- ◆ RISC II与几种CISC处理机的一次过程调用的平均开销比较

机器	执行指令条数	执行时间(us)	访问存储器次数
VAX-11	5	26	10
PDP-11	19	22	15
MC68000	9	19	12
RISC II	6	2	0.2

指令流执行次序调整

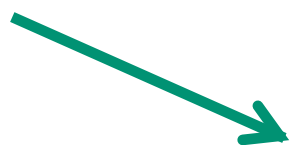
- ◆ 目的：**消除数据相关**，使原来可能存在的数据依赖消除，从而使流水线的运行效率达到最高。
- ◆ 方法：通过寄存器置换、编译优化的方法。

ADD R3, R1, R2 ; R1+R2->R3

ADD R5, R3, R4

MUL **R3**, R6, R7

MUL R9, **R3**, R8



ADD R3, R1, R2

MUL **R10**, R6, R7

ADD R5, R3, R4

MUL R9, **R10**, R8

操作类型小结

- ◆ 以下指令类型使用频度最高，指令系统应该支持这些类型的指令
 - load,
 - store,
 - add,
 - subtract,
 - move register-register,
 - and,
 - shift,
 - compare equal,
 - compare not equal,
 - branch,
 - jump,
 - call,
 - return;

2.5 操作数的类型和大小

- ◆ **数据表示**：计算机硬件能够直接识别、指令集可以直接调用的数据类型。
 - 所有数据类型中最常用、相对比较简单、用硬件实现比较容易的几种。
- ◆ **数据结构**：由软件进行处理和实现的各种数据类型
 - 研究：这些数据类型的逻辑结构与物理结构之间的关系，并给出相应的算法。
- ◆ 系统结构设计者要解决的问题：如何确定数据表示？
- ◆ 基本原则
 - (1) 有利于缩短程序的运行时间或CPI
 - (2) 有利于减少CPU与RAM之间的数据通信量
 - (3) 指令本身的通用性或利用率

2.5 操作数的类型和大小

- ◆ 表示操作数类型的方法有两种
 - 由指令中的**操作码**指定操作数的类型。
 - 带**标志符**的数据表示。给数据加上标识，由数据本身给出操作数类型。
 - 优点：简化指令集，可由硬件自动实现一致性检查和类型转换，缩小了机器语言与高级语言的语义差距，简化编译器等。
 - 缺点：由于需要在执行过程中动态检测标志符，动态开销比较大，所以采用这种方案的机器很少见。
- ◆ 操作数的大小：操作数的位数或字节数。
 - 主要的大小：字节（8位）、半字（16位）
字（32位）、双字（64位）

访问不同操作数大小的频度

操作数大小	访问频度	
	整型平均	浮点平均
字节	7%	0%
半字	19%	0%
单字	74%	31%
双字	0%	69%

SPEC基准程序

- ◆ 基准程序对单字和双字的数据访问具有较高的频度
 - 一台32位的机器应该支持8、16、32位整型操作数以及32位和64位的IEEE 754标准的浮点操作数。

2.6 MIPS指令系统结构

- ◆ MIPS64的一个子集，简称为MIPS
- ◆ MIPS的寄存器
 - 32个64位通用寄存器（GPRs）
 - R0, R1, ..., R31, 也被称为整数寄存器
 - R0的值永远是0
 - 32个64位浮点数寄存器（FPRs）
 - F0, F1, ..., F31
 - 用来存放32个单精度浮点数（32位），也可以用来存放32个双精度浮点数（64位）。
 - 存储单精度浮点数（32位）时，只用到FPR的一半，其另一半没用
 - 一些特殊寄存器
 - 它们可以与通用寄存器交换数据。
 - 例如，浮点状态寄存器用来保存有关浮点操作结果的信息

2.6.2 MIPS的数据表示

◆ 整数

- 字节（8位） 半字（16位）
- 字（32位） 双字（64位）

◆ 浮点数

- 单精度浮点数（32位）
- 双精度浮点数（64位）

◆ 字节、半字或者字在装入64位寄存器时，

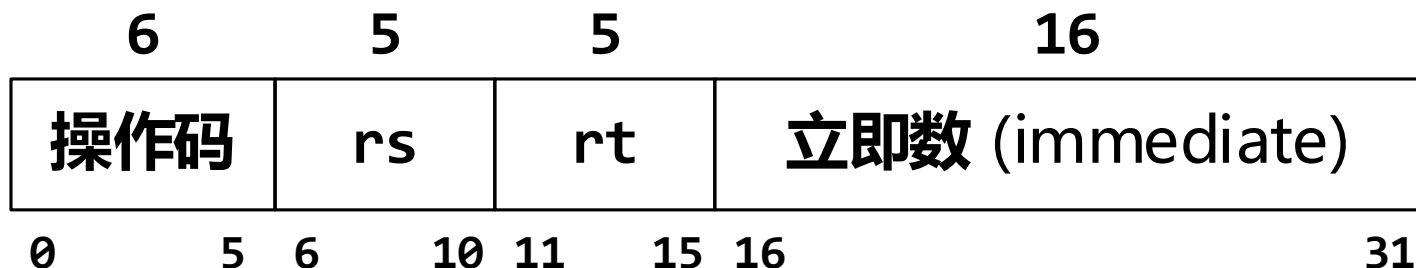
- 用零扩展或者用符号位扩展来填充该寄存器的剩余部分。
- 装入以后，对它们将按照64位整数的方式进行运算。

2.6.3 MIPS的数据寻址方式

- ◆ **立即数寻址与偏移量寻址**
 - 立即数字段和偏移量字段都是16位的。
- ◆ 寄存器间接寻址是通过把0作为偏移量来实现的
- ◆ 16位绝对寻址是通过把R0（其值永远为0）作为基址寄存器来完成的
- ◆ MIPS的存储器是按字节寻址的，地址为64位
- ◆ 所有存储器访问都必须是边界对齐的

2.6.4 MIPS的指令格式

- ◆ 寻址方式编码到操作码中
- ◆ 所有的指令都是32位的
- ◆ 操作码占6位
- ◆ 3种指令格式：**I类**，**R类**，**J类**
- ◆ **I类指令**
 - 包括所有的load和store指令、立即数指令、分支指令、寄存器跳转指令、寄存器链接跳转指令。
 - 立即数字段为16位，用于提供立即数或偏移量。



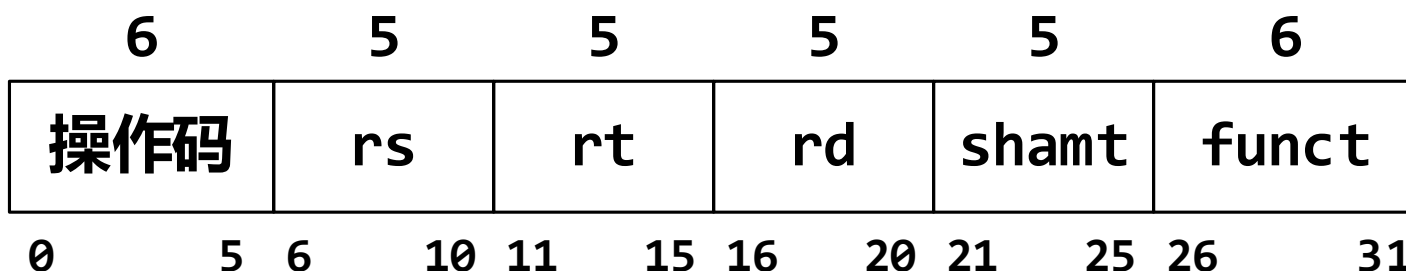
具体的I类指令

- ◆ load指令
 - 访存有效地址: $\text{Regs}[\text{rs}] + \text{immediate}$
 - 从存储器取来的数据放入寄存器rt
- ◆ store指令
 - 访存有效地址: $\text{Regs}[\text{rs}] + \text{immediate}$
 - 要存入存储器的数据放在寄存器rt中
- ◆ 立即数指令
 - $\text{Regs}[\text{rt}] \leftarrow \text{Regs}[\text{rs}] \text{ op } \text{immediate}$
- ◆ 分支指令
 - 转移目标地址: $\text{PC} + (\text{imm} \ll 2)$
 - 要判断的数据放在寄存器rt中, `beq $rs, $rt, imm`
- ◆ 寄存器跳转、寄存器跳转并链接
 - 转移目标地址为 $\text{Regs}[\text{rs}]$

2.6.4 MIPS的指令格式

◆ R类指令

- 包括ALU指令、专用寄存器读/写指令、move指令等。
- ALU指令
 - $\text{Regs}[\text{rd}] \leftarrow \text{Regs}[\text{rs}] \text{ funct } \text{Regs}[\text{rt}]$
 - func为具体的运算操作编码
 - `add $rd, $rs, $rt` ; $R[\$rd] \leftarrow R[\$rs] + R[\$rt]$



2.6.4 MIPS的指令格式

◆ J类指令

- 包括跳转指令、跳转并链接指令、自陷指令、异常返回指令。
- 在这类指令中，指令字的低26位是偏移量，它与PC值相加形成跳转的地址。
- 跳转指令使用伪绝对地址，例如
 - `j address ; PC <- {(PC+4)[0:3], address, 00}`



2.6.5 MIPS的操作

- ◆ MIPS指令可以分为四大类
 - load和store、ALU操作、分支与跳转、浮点操作
- ◆ 符号的意义
 - $x \leftarrow_n y$: 从y传送n位到x
 - $x, y \leftarrow z$: 把z传送到x和y
- ◆ 下标: 表示字段中具体的位;
 - 对于指令和数据, 按从最高位到最低位(即从左到右)的顺序依次编号, **最高位为第0位**, 次高位为第1位, 依此类推。
 - 下标可以是一个数字, 也可以是一个范围。例如:
 - $\text{Regs}[R4]_0$: R4的符号位;
 - $\text{Regs}[R4]_{56..63}$: R4的最低字节

2.6.5 MIPS的操作

- ◆ Mem：表示主存；
 - 按字节寻址，可以传输任意个字节。
- ◆ 上标：用于表示对字段进行复制的次数。
 - 例如： 0^{32} ：一个32位长的全0字段
- ◆ 符号##：用于两个字段的拼接，并且可以出现在数据传送的任何一边。
 - 举例：R6、R8：64位的寄存器，则
 - $\text{Regs}[\text{R8}]_{32..63} \leftarrow_{32} (\text{Mem}[\text{Regs}[\text{R6}]]_0)^{24} \text{## Mem}[\text{Regs}[\text{R6}]]$
 - 表示的意义是：
 - 以R6的内容作为地址访问内存，得到的字节按符号位扩展为32位后存入R8的低32位，R8的高32位（即 $\text{Regs}[\text{R8}]_{0..31}$ ）不变。

load和store指令



指令举例	指令名称	含义
LD R2, 20(R3)	装入双字	$\text{Regs}[R2] \leftarrow_{64} \text{Mem}[20+\text{Regs}[R3]]$
LW R2, 40(R3)	装入字	$\text{Regs}[R2] \leftarrow_{64} (\text{Mem}[40+\text{Regs}[R3]])_0^{32} \text{##}$ $\text{Mem}[40+\text{Regs}[R3]]$
LB R2, 30(R3)	装入字节	$\text{Regs}[R2] \leftarrow_{64} (\text{Mem}[30+\text{Regs}[R3]])_0^{56} \text{##}$ $\text{Mem}[30+\text{Regs}[R3]]$
LBU R2, 40(R3)	装入无符号字节	$\text{Regs}[R2] \leftarrow_{64} 0^{56} \text{##} \text{Mem}[40+\text{Regs}[R3]]$
LH R2, 30(R3)	装入半字	$\text{Regs}[R2] \leftarrow_{64} (\text{Mem}[30+\text{Regs}[R3]])_0^{48} \text{##}$ $\text{Mem}[30+\text{Regs}[R3]] \text{##} \text{Mem}[31+\text{Regs}[R3]]$
L.S F2, 60(R4)	装入单精度浮点数	$\text{Regs}[F2] \leftarrow_{64} \text{Mem}[60+\text{Regs}[R4]] \text{##} 0^{32}$
L.D F2, 40(R3)	装入双精度浮点数	$\text{Regs}[F2] \leftarrow_{64} \text{Mem}[40+\text{Regs}[R3]]$
SD R4, 300(R5)	保存双字	$\text{Mem}[300+\text{Regs}[R5]] \leftarrow_{64} \text{Regs}[R4]$
SW R4, 300(R5)	保存字	$\text{Mem}[300+\text{Regs}[R5]] \leftarrow_{32} \text{Regs}[R4]$
S.S F2, 40(R2)	保存单精度浮点数	$\text{Mem}[40+\text{Regs}[R2]] \leftarrow_{32} \text{Regs}[F2]_{0..31}$
SH R5, 502(R4)	保存半字	$\text{Mem}[502+\text{Regs}[R4]] \leftarrow_{16} \text{Regs}[R5]_{48..63}$

ALU指令

- ◆ 寄存器-寄存器型（RR型）指令或立即数型
- ◆ 算术和逻辑操作：加、减、与、或、异或和移位等

指令举例	指令名称	含义
DADDU R1, R2, R3	无符号加	$\text{Regs}[R1] \leftarrow \text{Regs}[R2] + \text{Regs}[R3]$
DADDIU R4, R5, #6	加无符号立即数	$\text{Regs}[R4] \leftarrow \text{Regs}[R5] + 6$
LUI R1, #4	把立即数装入到一个字的高16位	$\text{Regs}[R1] \leftarrow 0^{32} \text{ ## } 4 \text{ ## } 0^{16}$
DSLL R1, R2, #5	逻辑左移	$\text{Regs}[R1] \leftarrow \text{Regs}[R2] \ll 5$
DSLT R1, R2, R3	置小于	$\text{If } (\text{Regs}[R2] < \text{Regs}[R3])$ $\text{Regs}[R1] \leftarrow 1 \text{ else } \text{Regs}[R1] \leftarrow 0$

- ◆ R0的值永远是0，它可以用来合成一些常用的操作。

- ◆ 例如：

- DADDIU R1, R0, #100 ; 给寄存器R1装入常数100
- DADD R1, R0, R2 ; 把寄存器R2中的数据传送到寄存器R1

2.6.6 MIPS的控制指令

- ◆ 由一组**跳转**和一组**分支**指令来实现控制流的改变
- ◆ 典型的MIPS控制指令

指令举例	指令名称	含义
J name	跳转	$PC_{36..63} \leftarrow \text{name} \ll 2$
JAL name	跳转并链接	$\text{Regs}[R31] \leftarrow PC+4$; $PC_{36..63} \leftarrow \text{name} \ll 2$; $((PC+4) - 2^{27}) \leq \text{name} < ((PC+4) + 2^{27})$
JALR R3	寄存器跳转并链接	$\text{Regs}[R31] \leftarrow PC+4$; $PC \leftarrow \text{Regs}[R3]$
JR R5	寄存器跳转	$PC \leftarrow \text{Regs}[R5]$
BEQZ R4, name	等于零时分支	if ($\text{Regs}[R4] == 0$) $PC \leftarrow \text{name}$; $((PC+4) - 2^{17}) \leq \text{name} < ((PC+4) + 2^{17})$
BNE R3, R4, name	不相等时分支	if ($\text{Regs}[R3] \neq \text{Regs}[R4]$) $PC \leftarrow \text{name}$; $((PC+4) - 2^{17}) \leq \text{name} < ((PC+4) + 2^{17})$
MOVZ R1, R2, R3	等于零时移动	if ($\text{Regs}[R3] == 0$) $\text{Regs}[R1] \leftarrow \text{Regs}[R2]$

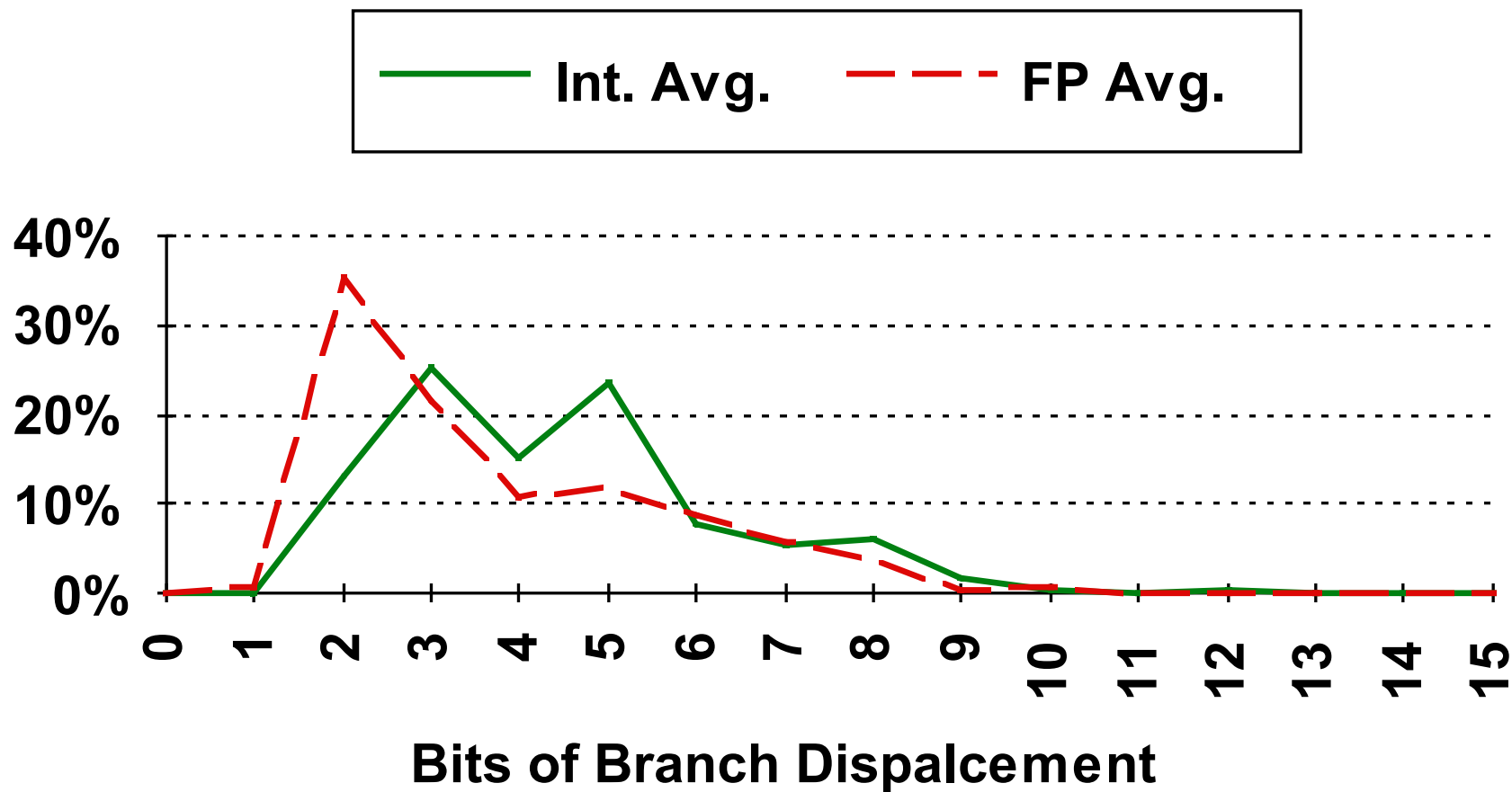
跳转指令

- ◆ 根据跳转指令确定目标地址的方式不同以及跳转时是否链接，可以把跳转指令分成4种。
- ◆ 确定目标地址的方式
 - 把指令中的26位偏移量左移2位（因为指令字长都是4个字节）后，替换程序计数器的低28位。
 - 间接跳转：由指令中指定的一个寄存器来给出转移目标地址。
- ◆ 跳转的两种类型
 - 简单跳转：把目标地址送入程序计数器。
 - 跳转并链接：把目标地址送入程序计数器，把返回地址（即顺序下一条指令的地址）放入寄存器R31。

分支指令（条件转移）

- ◆ 分支条件由指令确定。
 - 例如：测试某个寄存器的值是否为零
- ◆ 提供一组比较指令，用于比较两个寄存器的值。
 - 例如：“置小于”指令
- ◆ 有的分支指令可以直接判断寄存器内容是否为负，或者比较两个寄存器是否相等。
- ◆ 分支的目标地址。
 - 由16位带符号偏移量左移两位后和PC相加的结果来决定
- ◆ 浮点条件分支指令：通过测试浮点状态寄存器来决定是否进行分支。

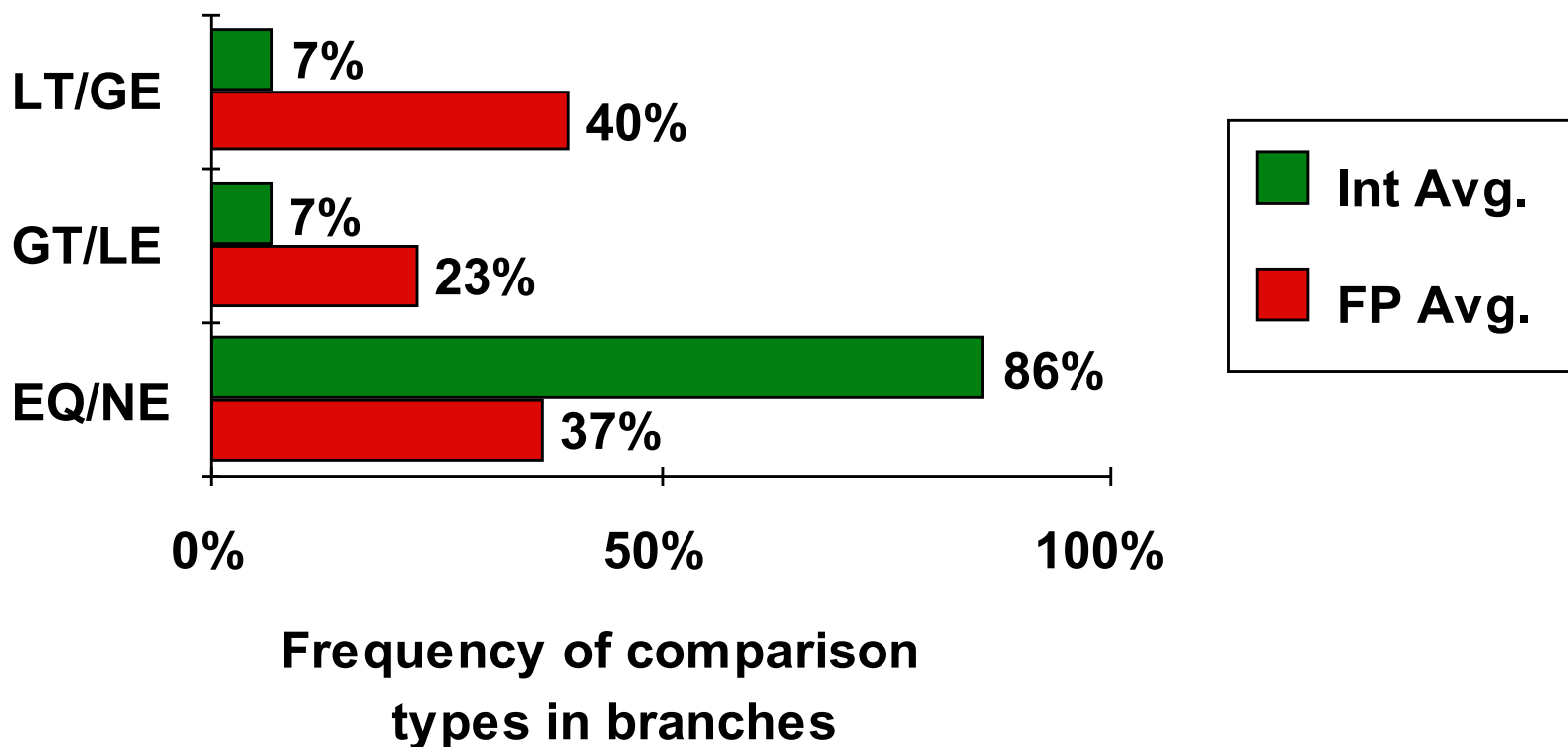
分支指令（条件转移）



- ◆ 25% of integer branches are 2 to 4 instructions

分支指令（条件转移）

- ◆ At least 8 bits suggested (± 128 instructions)
- ◆ Compare Equal/Not Equal most important for integer programs (86%)



2.6.7 MIPS的浮点操作

- ◆ 由操作码指出操作数是单精度(SP)或双精度(DP)
 - 后缀S: 表示操作数是单精度浮点数
 - 后缀D: 表示是双精度浮点数
 - MOV.S MOV.D
- ◆ 浮点操作
 - 包括加、减、乘、除, 分别有单精度和双精度指令。
 - ADD.S ADD.D SUB.S SUB.D
- ◆ 浮点数比较指令
 - 根据比较结果设置浮点状态寄存器中的某一位, 以便于后面的分支指令BC1T(若真则分支)或BC1F(若假则分支)测试该位, 以决定是否进行分支。